

Saurabh's Learning Roadmap — eYRC 2026-27

Team eYRC#4817 · Khoj-o-Drone (primary) · PacBot (secondary)

Contents (click to jump; the sidebar has bookmarks too)

1. Where you are starting from	M0 — The big picture
2. The deal: what Claude does vs what you do	M1 — How a computer sees
3. How every module works — the 5-step loop	M2 — How a drone flies
4. Plain-words dictionary	M3 — Feedback control & PID
5. Your three questions (answered 2026-09-13)	M4 — ROS 2: just enough for Tasks 1B/1C
6. The map	M5 — MuJoCo and the Swift Pico simulator
Stage 2 (real hardware) — only if selected, from November 2026	M6 — KD Task 1B: tune the altitude hold
7. Learning log — writing it down counts as explaining it	M7 — KD Task 1C: hold position in x and y
Answers	P1 — MQTT: how PacBot's programs talk
	P2 — Path planning
	P3 — Wall following with PID in MuJoCo
	L1 — After Task 1: modelling, stability and LQR
	T2 — After Task 1: Task 2 preview

Version 3 · rebuilt 2026-09-16 from all Khoj-o-Drone + PacBot learning material and both Task 1 briefs. Click the ☐ boxes to tick them and type into the log tables at the end; Document Viewer saves both into this PDF (Ctrl+S). To change the plan itself, tell Claude "update my roadmap".

1. Where you are starting from

You already know	New to you
Linux and its commands · C++ · breadboard circuits · introductory Python (practising daily)	OpenCV · ROS 2 · MuJoCo · MQTT · PID · path planning · how a drone flies

The good news: Linux and C++ are what trips most beginners, and you already have them. Most of what's new is *ideas*: pictures as numbers, feedback, programs sending messages to each other, searching a maze, physics in a simulator.

Task 1 is out for both themes (16 Sep 2026). Until Task 2/3 the team must carry both themes:

Theme	Part	What it is	You write code or tune?	Marks	Module
Khoj-o-Drone	1A	Find survivors in a photo, name their grid points	write (OpenCV)	20	M1
Khoj-o-Drone	1B	Make the drone hold a fixed height	tune 3 gains	40	M6
Khoj-o-Drone	1C	Make it hold its place (x and y too)	tune 6 more gains	40	M7
PacBot	1A	Drive a grid maze: 2 pellets, then an exit, over MQTT	write (Python)	35	P2
PacBot	1B	Wall-follow a 3D MuJoCo maze with a PID	write (Python + PID)	65	P3

Five subtasks is too much for one person. Agree a split with the team. A proposal to discuss, not a decision: Mahesh + Saurabh on KD 1A, Gauri on KD 1B/1C tuning, Parth + Saurabh on PB 1A, Gauri + Saurabh on PB 1B. Whatever the split, **everyone still does the TALK and TEACH steps for every module**, because the team picks one theme later and everyone must understand both.

2. The deal: what Claude does vs what you do

Claude does (not worth your learning time)	You do (this is the learning)
Installing, building, library fixes, workspace setup (all done for Task 1)	Predict what will happen before anything runs
Launching simulators, brokers and tuners in the right order	Choose what to change, one thing at a time
Measuring tools: pixel picker, colour sliders, MQTT traffic viewers, plots	Watch and notice what actually happened
Test benches, format and coding-standard checkers, score estimators	Explain why it happened, to Claude or a teammate
Packaging checks: zip structure, file names, "no GUI calls"	Write the brain code (1A pipelines, the PacBot planner and PID) and choose every gain (KD 1B/1C)
Screen-recording test before real submissions	Teach it to someone, then decide what to try next

Ground rules for Claude in every learning session

1. Define every new word before using it. No unexplained acronyms.
2. Ask me to **predict** before running any experiment.
3. When I'm explaining, don't correct me mid-way. Ask "why?" until I get stuck.
4. Change one thing at a time.
5. **For graded tasks, give hints and questions, never finished solution code.** Review my code after I've written it.
6. Be token-conscious: short answers, targeted reads, ask before big optional work.

Why rule 5 matters twice: writing it yourself is how you learn it, *and* e-Yantra runs every submission through plagiarism software. The PacBot launchers are also encrypted; **never try to open or inspect them**, because that counts as tampering.

e-Yantra coding standard (every submitted file): a header block with Team ID 4817, theme, authors, filename, functions and globals · a Purpose / Input Arguments / Returns / Example call block under every function · descriptive names (no *a*, *b*, *temp*) · comments on tricky parts. **For PacBot, never rename or restructure the boilerplate's functions.** Add comments only.

How to start a session: say "*Roadmap P2, experiment P2-c*". Claude sets everything up, and you drive.

3. How every module works — the 5-step loop

1. **WATCH:** a *video ladder*. Rung ① is short, rung ② covers the same idea in more depth, later rungs go long. Stop once it clicks.
2. **TALK:** explain the idea to Claude in your own words. Claude keeps asking "why?".
3. **DO:** hands-on experiments on a **Predict → Change one thing → Notice → Explain** card.
4. **TEACH:** explain it to a teammate in 5 minutes. If you need notes, go back to TALK.
5. **CHECK:** answer the check questions without looking anything up.

4. Plain-words dictionary

Image words (KD Task 1A)

Word	What it actually is	Everyday picture	How deep
Pixel	One dot of an image; a colour pixel is 3 numbers from 0-255	One tile in a mosaic	Deep (M1)
OpenCV (cv2)	The library of image tools you call from Python	A toolbox for pictures	Deep (M1)
BGR	OpenCV stores colour as Blue-Green-Red, not RGB	An address written backwards	Know — bites everyone once
HSV	Hue (which colour), Saturation (how vivid), Value (how bright). OpenCV hue is 0-179 , not 0-359	A paint name + "light/dark"	Deep (M1)
Mask / threshold	Black-and-white image: white where a rule is true	A stencil	Deep (M1)
Contour	The outline of one white blob in a mask	Tracing a shape with a pencil	Deep (M1)
Centroid / moments	The centre point of a shape	The balance point of a cut-out	Understand (M1)
ArUco marker	A square black-and-white code with an ID	A tiny QR code	Understand (M1)
Perspective transform	Maths that straightens a photo taken at an angle using 4 points	Flattening a sideways photo of a poster	Deep (M1)

Control and drone words (KD 1B/1C, PacBot 1B)

Word	What it actually is	Everyday picture	How deep
Setpoint · Error	Where you want to be; error = setpoint – where you are	Destination, and distance left	Deep
Open / closed loop	Acting without / with measuring the result	Walking eyes closed / eyes open	Deep (M3)
PID	Feedback rule: P reacts to error now, I removes leftover error, D brakes when approaching fast	Parking a car	Deep — 4 of the 5 subtasks
Overshoot · steady-state error	Going past the target · settling slightly off it	Braking late · parking 10 cm short	Deep (M3)
Integral windup	The I term piling up while stuck, then overshooting	Pushing a stuck door until it flies open	Understand (M3)
Throttle · Roll · Pitch · Yaw	Up/down · tilt left/right · tilt forward/back · spin	—	Deep (M2)
Tilt → sideways motion	Tilting splits thrust; sideways acceleration $\approx g \times \text{tilt angle}$	Pushing a tilted phone up	Deep (M2, M7)
Cascade control	Position loop → angle loop → motor speeds, each feeding the next	A manager → supervisor → worker	Understand (M2)
RC values 1000-2000	Drone command numbers; 1500 = neutral	Joystick position	Understand
ToF sensor	Time-of-Flight: measures distance by timing light	A bat's echo, with light	Understand (P3)

Word	What it actually is	Everyday picture	How deep
IMU (gyro + accelerometer)	Measures turning rate and acceleration	Your inner ear	Understand (P3)
State space · LQR	A matrix model of a system · an "optimal" controller built on it	—	Later (L1)

Software and messaging words

Word	What it actually is	Everyday picture	How deep
ROS 2	Not an OS: a messaging system + tools for robot programs	Group chats between programs	Understand (M4)
Node · Topic · Message	A running program · a named channel · one packet on it	Person · group name · one post	Understand (M4)
Bag file	A recording of topic messages; your KD 1B/1C submission	A voice recorder for robot chat	Know (M4, M6)
MQTT	Lightweight publish/subscribe for devices, through a central broker	A post office	Deep (P1)
Broker (Mosquitto)	The MQTT server that forwards every message; runs on your laptop	The post office itself	Know (P1)
Wildcards + #	+ = one topic level, # = everything below	"Any room" / "whole house"	Know (P1)
QoS 0/1/2	At most once / at least once / exactly once delivery	Postcard / registered post / signed-for	Understand (P1)
Retained message	The broker keeps the last value for newcomers	A notice board	Know (P1, P2)
MuJoCo	The physics simulator both themes use	A game with real physics	Understand (M5, P3)
Graph · node · edge	Maze cells joined where you can move	Cities and roads	Deep (P2)
BFS · DFS · Dijkstra · A*	Ways to search a graph; A* = cost so far + estimate to goal	Search patterns in a maze	Deep (P2)
Heuristic	A quick estimate of distance to the goal	"It's roughly that way"	Deep (P2)

5. Your three questions (answered 2026-09-13)

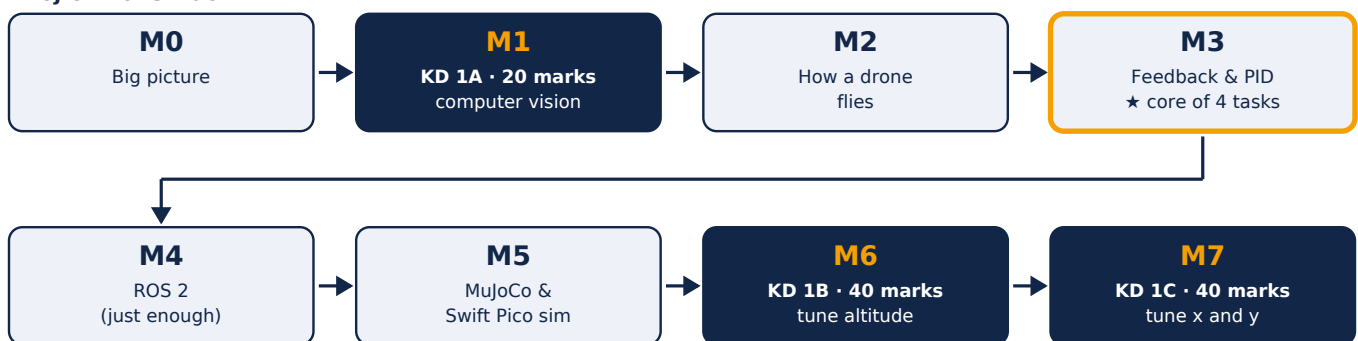
Why a circle? e-Yantra's bonus task asked for one, and the circle itself doesn't matter. It is the smallest exercise that **sends movement commands and reads back where the robot is**, which is the same loop as every Task 1 subtask.

Why was the radius predefined? The portal said diameter 2.0 (turtlesim units, not metres). Its picture showed about 8 units, so Claude made the radius changeable.

The drone's size? The simulated Swift Pico (from [drone.xml](#)): 47 × 47 × 11 cm body, motors ~27 cm from centre, **1.5 kg (weight 14.7 N), max 5.47 N thrust per motor (21.9 N total)**, 31.3 cm marker, overhead camera 20 m up with a 60° view, physics every 0.005 s. **The real kit's size is unknown** until Stage 2.

6. The map

Khoj-o-Drone track



PacBot track (MQTT, planning, wall-following PID)



After Task 1



Suggested pace, about 1.5 h a day, as a guess. The Task 1 deadline isn't known yet; the schedule suggests ~4 weeks (mid-October). If it's earlier, the plan gets squeezed, not the understanding: skip the deep video runs first.

Week (guess)	Modules	Goal
Sep 17–23	M0 · M1 · P1 (evenings)	KD 1A ready for Gauri; MQTT understood
Sep 24–30	P2 · M2	PB 1A ready; feel how a drone moves
Oct 1–7	M3 · M4 + M5 (just enough) · M6	PID understood; KD 1B tuned and recorded
Oct 8–14	M7 · P3	KD 1C and PB 1B done
After Task 1	L1 · T2	LQR foundations; Task 2 preview

Goal: know what both themes build, and which tool does which job.

WATCH

- ☐ ① [eYRC 2026-27 Official Launch](#) (e-Yantra). *Notice:* Stage 1 simulation → Stage 2 hardware.
- ☐ ② The portal's **Quadcopter Control** page: the "Quadcopter Operation" diagram (overhead WhyCon camera → laptop PID/LQR → RC transmitter → drone's attitude controller → motors), and the PacBot Task 1 topic maps (simulator ↔ broker ↔ your code).

TALK

Claude asks you to trace one command through each system. KD: your gains → controller program → `/drone_command` → attitude controller → motors → camera → back. PacBot: your code → broker → simulator → broker → your code. Where is the feedback in each?

DO

- ☐ **M0-a** On paper, draw both theme missions as boxes and arrows. Label each box with the skill it needs (OpenCV, PID, MQTT, path planning, ROS 2).
- ☐ **M0-b** Circle what each of the five Task 1 subtasks practises. Which skills do both themes share? (Hint: PID and path planning.)

TEACH

a 5-minute "what we're building, and what Task 1 asks" talk to Parth or Mahesh.

CHECK

1. In one sentence each: what do OpenCV, ROS 2, MQTT, MuJoCo and PID do in our project?
2. Why does the drone simulator mirror the real hardware setup?
3. Which Task 1 subtasks are *tuning* and which are *writing code*?

Goal: a program that turns one arena photo into survivor names like D2, and you understand every line.

Specs and traps: [context/09-task1-overview-and-1a.md](#). Starter + photo:

`~/pico_ws/src/swift_pico/scripts/`. The portal's image-processing pipeline (acquire → clean → segment → measure → decide) is exactly the 7 steps below.

WATCH (each rung assumes the one before)

- ☐ ① [Digital Images — Computerphile](#). *Notice:* an image is a grid of numbers.
- ☐ ② The portal's **Image Processing Basics** page (BGR caution, HSV table, hue 0-179), then [OpenCV: Changing Colorspaces](#). *Notice:* the portal lists red as 0-60°, but red also wraps around at 330-360°.
- ☐ ③ [OpenCV Course — Full Tutorial with Python](#) (freeCodeCamp, the portal's pick). **Only:** reading images, drawing, colour spaces, thresholding, contours.
- ☐ ④ [Warp Perspective / Bird View](#) (short). *Notice:* 4 points in, 4 points out, order matters.
- ☐ ⑤ Deep: [Computing Homography](#) (Shree Nayar).
- ☐ ⑥ Read: [Detecting ArUco markers with OpenCV and Python](#). It uses the **old** ArUco commands, which match **your OpenCV 4.5.4**. The portal's list has no ArUco page, and newer docs use `ArucoDetector`, which **doesn't exist on your machine**.

Keep open while coding: the portal's OpenCV Python page (the `inRange` example) · [Contours](#) · [Contour Features](#) · [Geometric Transformations](#) · [Morphological Transformations](#) (cleaning speckles out of masks).

TALK

- Answer the portal's 4 observation questions out loud, before any code.
- Why must the photo be straightened before naming intersections?
- Why does red need *two* HSV ranges?
- Recognise each corner by the marker's ID or by its position in the photo? What breaks in each case?

DO — you write every pipeline step; Claude supplies tools and checks

- ☐ **M1-a Look first (no code).** Open `image_1.jpg`, count the survivors, and *guess* each name. That's your answer key.
- ☐ **M1-b Pixel detective** ([learning/tools/pixel_detective.py](#)). Predict the B-G-R numbers of red, yellow, blue, black and green **before** clicking. Which HSV number stays steady across a shaded yellow?
- ☐ **M1-c Step 1 — markers** (old ArUco API). *Experiment:* Claude paints over one marker. Does your "missing marker" message fire?
- ☐ **M1-d Step 2 — straighten to 900 × 900.** On paper, pick 4 of the 16 marker corners. *Experiment:* swap two destination points; predict the picture.
- ☐ **M1-e Steps 3-4 — grid and names, on paper first.** How many pixels per cell? Where is C2? (*Answers on the last page.*) Then draw your green grid and compare with the portal figures.
- ☐ **M1-f Step 5 — colour masks** ([learning/tools/hsv_tuner.py](#)). Keep only red, then only yellow. What sneaks in? Count your contours against M1-a.
- ☐ **M1-g Step 6 — centres from moments.** How can a shape have zero area, and what should your code do?
- ☐ **M1-h Step 7 — nearest intersection + edge guard + composite debug picture.**
- ☐ **M1-i Results file + coding standard.** You write both. Claude's checker compares the file character by character and checks your comment blocks. (Ask on the forum about the 3-line vs 4-line question the two portal pages disagree on.)

- ☐ **M1-j Stress test.** Claude's test bench tilts, rotates, darkens and blurs the arena. Fix what breaks without special-casing `image_1.jpg`.
- ☐ **M1-k Submission dry run.** `KD_4817_task1a.py`, no `imshow/waitKey/input()`, zip the file itself, hand it to Gauri.

TEACH walk Mahesh through your composite debug picture, step by step.

CHECK

1. Why HSV instead of BGR? 2. What does a perspective transform need, and what does wrong point order do? 3. Why nearest intersection, not "which cell"? 4. Why no `imshow` in the submission?

M2 How a drone flies

3 days

Goal: *feel* why a quadcopter moves the way it does.

WATCH

- ☐ ① [Drone flight physics in under 2 minutes](#) (Dronology, 1:38, the portal's pick). *Notice:* which motors speed up for each move.
- ☐ ② The portal's **Quadcopters** page: forces F1-F4, motor torques, 4 DOF, sensors. *Notice:* roll = rotation about **x** → movement along **y**; pitch = rotation about **y** → movement along **x**.
- ☐ ③ [Introduction to Quadcopters](#) (e-Yantra tech talk). **Watch only ~12-26 min** (dynamics, why diagonal motors spin the same way) **and ~42-48 min** (the control cascade: position → angle → motor speed). That cascade *is* your Task 1B/1C setup.
- ☐ ④ Optional: [Drone Simulation and Control playlist](#) (MATLAB).

TALK

- Why can't a quadcopter move sideways without tilting?
- When it tilts, what happens to its height, and why?
- What if all four propellers spun the same way?

DO

- ☐ **M2-a Phone on your palm (no computer).** Push a flat phone straight up; then tilt it 20° and push along its own "up". *Notice:* you move sideways and must push harder to keep height.
- ☐ **M2-b Pencil and paper with \$5's numbers.** What % of full thrust does hovering need? At what tilt can the Pico no longer hold height? (*Answers on the last page.*)
- ☐ **M2-c Throttle guessing game — you type the commands.** Claude launches the simulator; you arm with the portal's `ros2 topic pub /drone_command ... (throttle 1500, rc_aux4: 2000)`, then try 1600. *Predict* first. *Notice:* does 1500 really hover forever? Ctrl+C between commands; disarm only near the ground.
- ☐ **M2-d Notice the numbering.** The portal's motor diagram numbers the motors differently from `drone.xml`. Why does that matter for real hardware?

TEACH explain to Gauri why tilting makes a drone move *and* sink.

CHECK

1. Which channel moves the drone forward, and why is that "indirect"?
2. Why disarm only near the ground?
3. Draw the cascade from the e-Yantra talk. Which box do you tune in Task 1B?

Goal: feel what P, I and D each do, on things you can break freely.

Why it matters: KD 1B + 1C + PB 1B = 145 of the 200 Task 1 marks are PID. The portal says you'll later **implement** PID and LQR/LQI yourselves. Gauri studied this in EE, so learn with her.

WATCH (do M3-a to M3-c *before* the videos)

- ☐ ① The portal's **Introduction to Control Systems** (open vs closed loop), then [PID Control — A brief introduction](#) (7:44).
- ☐ ② [What Is PID Control? Part 1](#) (MATLAB) → [Simple Examples of PID Control](#) (13:10).
- ☐ ③ [Anti-windup, Part 2](#) · [Noise filtering, Part 3](#) · [A PID Tuning Guide, Part 4](#) (the most useful before tuning).
- ☐ ④ The portal's **P, PD & PID for Drones** page: P oscillates, PD leaves a steady-state error, PID settles on target. Then press Start on the portal's **ball-and-beam demo** and reproduce all three yourself.
- ☐ ⑤ [Manual and Automatic Tuning, Part 6](#) · [Important PID Concepts, Part 7](#) · big picture: [Everything You Need to Know About Control Theory](#).
- ☐ ⑥ Reference only: the portal's control book and the 46-video *Classical Control Theory* playlist. Dip in when a concept is stuck.

TALK

- Why doesn't P alone bring a *heavy* drone exactly to the target?
- Why does D stop the bouncing?
- What is integral windup? Give an everyday example.

DO

- ☐ **M3-a Balance a pen on your palm**, then while looking away. *Notice:* you use the tilt (P) and how fast it's tilting (D); delay makes it fall.
- ☐ **M3-b Eyes-closed walk** to a door 5 m away, then eyes open. Open vs closed loop.
- ☐ **M3-c Tap-water temperature**. The delay makes you over-correct back and forth.
- ☐ **M3-d Spot the bug**. The portal's PID page writes the I term as $I_{term} = (I_{term} + error) \cdot K_i$. Run 5 steps by hand with $K_i = 0.5$ and $error = 1$, then $K_i = 2$. What goes wrong, and what's the correct form? (*Answers on the last page.*)
- ☐ **M3-e Turtle gain sweep**. Turning gain 0.5, 6, 60, 200; predict each. Why doesn't the turtle overshoot? (It has no mass.)
- ☐ **M3-f A toy drone in 30 lines of plain Python**. Claude writes the physics loop and plot; **you write** `thrust = hover + Kp * error`, then add D, then I (Claude makes it heavier than your hover guess), then a 0.1 s sensor delay. Log every gain in §7.

TEACH P, I and D to Mahesh with the car-parking picture; Gauri checks the maths.

CHECK

1. What does each of P, I and D fix, and what breaks when each is too large?
2. Why did the turtle not bounce but the toy drone did?
3. What is integral windup?

M4 ROS 2: just enough for Tasks 1B/1C

2 days

Goal: run three terminals confidently, read a topic, and record a bag.

WATCH

- ☐ ① [What Is ROS2? – Framework Overview](#) (8:22, the portal's pick). *Notice:* nodes, topics, services, actions, parameters, **bag files**.
- ☐ ② The portal's **Turtlesim Resources** page. *Notice:* spawn, remap, action feedback.
- ☐ ③ Later / Task 2: the portal's official tutorial sequence (Beginner CLI → Client Libraries) and [Articulated Robotics: ROS Overview](#).

Keep open: eYRC 2026-27/Media/Learnings/ros2_cli_cheat_sheet.pdf (from 2019): `ros2 msg / ros2 srv` no longer exist in Humble (use `ros2 interface show`), and its bag examples leave out the word `bag`.

TALK

- If the node sending commands crashes, what does the robot do?
- Your KD controller reads the drone's position and sends commands. Which is the subscriber, and which the publisher?
- Why does every new terminal need `source ../setup.bash`, and why Ctrl+C, never Ctrl+Z?

DO — you type these

- ☐ **M4-a Turtlesim by hand.** Run the portal's commands yourself: `teleop`, `ros2 topic echo /turtle1/pose`, `/spawn` a second turtle, `rotate_absolute` with `--feedback`, remap `teleop` to `turtle2`.
- ☐ **M4-b Predict the graph** on paper, then compare with `rqt_graph`.
- ☐ **M4-c Kill the teleop mid-move.** The turtle stops after ~1 s. Read the few C++ lines in `turtle.cpp` that cause it.
- ☐ **M4-d Record and replay a bag.** `ros2 bag record` the turtle's pose while you drive, then `ros2 bag info` and `ros2 bag play`. This is the same skill as the KD 1B/1C submission.

TEACH publish/subscribe to Parth, using `rqt_graph` as proof.

M5 MuJoCo and the Swift Pico simulator

2 days

Goal: know what the simulator does, and trust it (or not) for the right things.

WATCH / READ

- ☐ ① The portal's MuJoCo pages + [MuJoCo Overview](#). Notice: the **model** (XML, fixed) vs the **data** (state, changes every step).
- ☐ ② [MuJoCo introductory tutorial \(Colab\)](#): run the cells and change the numbers.
- ☐ ③ The portal's **Swift Pico** page: topics, `SwiftMsgs` (int64 fields), arming. Its screenshots show old names; use `/drone_command` and `~/pico_ws`.
- ☐ ④ Optional: [MuJoCo Menagerie](#). Compare the Crazyflie 2 / Skydio X2 quadrotor models with the Pico.

TALK

name two things a simulator might get wrong compared with a real drone ("sim-to-real gap").

DO

- ☐ **M5-a Listen to the drone.** Claude launches the sim; you run `ros2 topic list`, `ros2 topic echo /whycode_node/markers`. Which number is height?
- ☐ **M5-b Explore the viewer panels:** pause, single-step, contacts, "Print data".
- ☐ **M5-c Drop test on paper.** How long does a 2 m drop take? ($t = \sqrt{2h/g}$) Then watch a disarm from 2 m and compare.

TEACH

model vs data to Mahesh (the recipe vs the dish while cooking).

M6 KD Task 1B: tune the altitude hold

40 marks

Goal: Throttle error inside ± 0.4 within 5 s, held for 10 s. **You choose every gain; no code.**

Full brief: [context/10-task1b.md](#). Setup is done and verified (library fixes, tuner, bag recording).

- ☐ **M6-a Read the brief with Claude.** In your own words: what is scored (hover 16 + finish bonus 8 + speed 16), and when does the clock start? Ask on the forum if unsure.
- ☐ **M6-b First bring-up — you run the 3 terminals:** `sim` → `ros2 run swift_pico task_1b_controller` (answer Y) → `ros2 launch pid_tune pid_tune_drone.launch.py`. *Predict* what the drone does with $K_p = K_i = K_d = 0$.
- ☐ **M6-c Know your tuner.** Sent value = integer $\times$ scale (defaults $K_p \times 0.03$, $K_i \times 0.008$, $K_d \times 0.6$). Changes are sent **only when you press -/+ or Enter**. Check it: does the drone react before you touch anything?
- ☐ **M6-d Tune by the portal's method**, using the tuner's live `/pos_error` graph: raise K_p until it holds but oscillates → back off → add K_d → small K_i . One change at a time; log every step in §7. *Notice:* do your M3 lessons hold?
- ☐ **M6-e Save and score.** Press Save Values (writes `~/pico_ws/src/swift_pico/src/pid_values.yaml`). Record a practice bag; Claude's self-scorer applies the marking rules to it.
- ☐ **M6-f Test recording (Claude helps):** 2-minute test with Ubuntu's built-in recorder. Your session is Wayland, so Kazam/SimpleScreenRecorder may record black.
- ☐ **M6-g Final run:** recorder on, terminal visible from the start → `sim` → controller, typing your gains (answer N) → `ros2 bag record -o task_1b /pos_error /whycode_node/markers ≥ 60 s` → zip the two files → Gauri uploads `KD_4817_task_1b.zip` + unlisted `KD_4817_Task_1b` video link.

TEACH show the team one wobble you fixed, and how you knew which gain caused it.

M7 KD Task 1C: hold position in x and y

40 marks

Goal: Throttle, pitch and roll errors all within ± 0.4 within 5 s, held for 10 s. Keep your 1B throttle gains.

Brief: [context/11-task1c.md](#).

- ☐ **M7-a The physics on paper (with Gauri).** The portal's Quadcopter Control "To Do" asks for the translational equations of motion. Its recipe (only gravity + motion energy) gives *free fall*, so what's missing? Add thrust tilted by the angle and linearize about hover. (*Answers on the last page.*)
- ☐ **M7-b Predict:** to move toward +x, which channel changes? What happens to height at the same moment?
- ☐ **M7-c Tune pitch and roll together** (same gains; the frame is symmetric) with [task_1c_controller](#) running. *Notice:* the drone sinks when it tilts, so throttle may need a touch-up.
- ☐ **M7-d Final run.** The portal order is sim → **tuner** → controller. Watch for the "gains only sent on click" trap; typing gains at the controller prompt avoids it. Record [task_1c](#) ≥ 60 s, [KD_4817_task_1c.zip](#), [KD_4817_Task_1c](#) video.

TEACH why x/y control is "indirect", using M2-a's phone trick.

P1 MQTT: how PacBot's programs talk

2 evenings

Goal: understand broker, topics, wildcards, QoS and retained messages well enough to debug PacBot Task 1 traffic. **Setup done:** Mosquitto broker running on your laptop (local only), paho-mqtt 2.1.0.

WATCH / READ

- ☐ ① The portal's **MQTT** page (the mqtt.org temperature diagram) + **MQTT Concepts** page (wildcards and QoS diagrams).
- ☐ ② HiveMQ MQTT Essentials: [Topics](#) · [QoS](#) · [Retained messages](#) · [Last Will](#).
- ☐ ③ The portal's **MQTT Tutorials** page (Python paho example).

TALK

- MQTT vs ROS 2: same pub/sub idea, so what's different? (Broker in the middle; untyped payload.)
- Why are PacBot 1A's pose and pellets *retained*, but commands not?
- **Security (your area):** why is `test.mosquitto.org` fine for "hello" but never for real data?

DO — you type everything, on the local broker

- ☐ **P1-a Hello broker.** Terminal 1: `mosquitto_sub -h localhost -t pacbot/test -v`; terminal 2: `mosquitto_pub -h localhost -t pacbot/test -m hi`.
- ☐ **P1-b Wildcards.** Subscribe to `home/+ / temperature` and `home/#`. *Predict* which of `home/kitchen/temperature` and `home/livingroom/humidity` each one receives, then publish both.
- ☐ **P1-c Retained.** Publish with `-r`, *then* start a new subscriber. *Notice:* it gets the value instantly. Now do it without `-r`.
- ☐ **P1-d Python.** Run the portal's paho subscriber + publisher (VERSION2 callbacks). Change the topic; break the topic name on purpose. What does "nothing happens" look like?

TEACH the post-office picture of MQTT to Parth, with P1-c as proof.

P2 Path planning

PacBot Task 1A · about 5 days · 35 marks

Goal: a Python planner that eats both pellets and exits, sending one command at a time.

Brief: `context/15-pacbot-task1.md`. Your code goes only inside `choose_command()` in `~/pacbot_ws/task1a/task_1a.py`. **Never rename the boilerplate's functions.**

WATCH / READ

- ☐ ① The portal's **Path Planning** page and its animations (BFS rings, DFS thread, Dijkstra, Greedy, A*). *Notice:* the only difference is **which frontier cell goes next**.
- ☐ ② [Dijkstra's Algorithm — Computerphile](#) (10 min) → [A* Search Algorithm — Computerphile](#) (14 min, builds on it).
- ☐ ③ The portal's PacBot Task 1A instructions and topic map (robot/pose, pellets/pose, bot/cmd, robot/cmd_vel).

TALK

- Why does BFS give the shortest path on this maze, but DFS doesn't?
- Why is a turn plus a move *two* commands here? What state must your code remember?
- Two pellets and two exits: in what order should you visit them, and how would you decide?

DO

- ☐ **P2-a By hand on paper.** On a 6×6 grid with a few walls, run BFS and DFS with a pencil. Count the cells each explores before reaching the goal.
- ☐ **P2-b Watch the traffic.** Claude starts the simulator; you run `mosquitto_sub -h localhost -t 'robot/pose' -t 'pellets/pose' -t 'bot/cmd' -v`.
- ☐ **P2-c Drive by hand.** `mosquitto_pub -h localhost -t robot/cmd_vel -m LEFT`, then `FRONT`, and so on. **Find out yourself:** does `LEFT` add +90° to yaw or −90°? What does `valid: false` look like when you drive into a wall?
- ☐ **P2-d Write BFS** in plain Python on a small hand-made grid first (daily Python practice), then on the real maze.
- ☐ **P2-e Turn logic.** From (current yaw, direction to next cell) → which command? Make a 4 × 4 table on paper, then code it.
- ☐ **P2-f Pellet order + exit.** Try both orders, and pick the shorter total route.
- ☐ **P2-g Submission dry run.** Coding-standard comments added (no renamed functions), Claude checks it. Recorder on → `./task_1a_launch --evaluate` → your code → `zip result.yaml + task_1a.py` as `PB#4817.zip` → Gauri uploads, plus the unlisted video.

TEACH explain to Parth why your bot never sends `FRONT` while facing a wall.

P3 Wall following with PID in MuJoCo

PacBot Task 1B · about 5 days · 65 marks

Goal: enter, follow the walls, exit, **with zero collisions**. Marks = 65 – 10 per collision, and 0 if it doesn't exit. **Builds directly on M3.**

WATCH / READ

- ☐ ① Revisit M3 ③ (tuning guide) and ④ (P / PD / PID for drones). This is the same controller on a wheeled robot.
- ☐ ② The portal's PacBot Task 1B instructions: [pacbot/sensors](#) (ToF fl/fr/sl/sr, gyro, accel, dt at ~500 Hz) and [pacbot/wheel_vel](#) (latest value wins).

TALK

- What is the *error* for wall following? (Desired side distance – measured side distance.)
- Why use `dt` in the I and D terms? Why use the gyro to finish a turn, rather than a timer?
- Why does "keep driving at the last command" make a missed message dangerous?

DO

- ☐ **P3-a Watch the sensors.** Claude starts the simulator; you `mosquitto_sub -t pacbot/sensors`. Push nothing yet. Which numbers change when you imagine the bot moving closer to a wall? Check once it moves.
- ☐ **P3-b P only.** You write the P-controller for side distance. *Notice:* the corridor wobble.
- ☐ **P3-c Add D, then a small I.** Count collisions in the launcher terminal after each change.
- ☐ **P3-d Corners.** Front ToF says "wall ahead" → turn using the gyro until 90° is reached. Log what overshoots.
- ☐ **P3-e Submission dry run.** Coding-standard comments, recorder on, `./task_1b_launch --evaluate`, your code, zip `result.json + task_1b.py` as `PB#4817.zip`.

TEACH

show Gauri your tuning log, and let her guess which gain you changed from the collision count.

L1 After Task 1: modelling, stability and LQR

Why: the portal says you'll **implement PID and LQR/LQI** on the drone later. Understand-level, paired with Gauri.

- ☐ ① The portal's **System Modelling** page ($F = ma \rightarrow$ Euler-Lagrange $\rightarrow$ state space) and **Model Examples** (point mass, spring-mass-damper, projectile, rotation).
- ☐ ② [Introduction to State-Space Equations](#) (14:12).
- ☐ ③ The portal's **Stability + Pendulum** pages, with [Introduction to System Stability and Control](#) $\rightarrow$ [Stability of Closed Loop Control Systems](#). **DO with Gauri:** the Jacobian of the hanging vs upright pendulum on paper. Which eigenvalues say "unstable"?
- ☐ ④ [3 Ways to Build a Model](#) $\rightarrow$ [What Is LQR?](#) $\rightarrow$ [Why the Riccati Equation Is Important](#) $\rightarrow$ the portal's LQR page + Murray notes (download from the portal).
- ☐ ⑤ [MuJoCo LQR Colab](#). Octave gets installed only when a task needs it.
- ☐ ⑥ Optional mindset: [A real control system — how to start designing](#) (27 min).

T2 After Task 1: Task 2 preview

- ☐ **Camera in ROS 2.** The portal's ROS 2-OpenCV page (`cv_bridge`, already working here). Note: the drone sim only publishes `/image_raw` when launched with `image_sink:=both`.
- ☐ **Pixels to metres.** Camera 20 m up, 60° view: how many cm of ground per pixel? Claude checks against `/camera_info`.
- ☐ **Searching an area.** Your P2 skills apply to a drone search grid.
- ☐ Optional long videos: [Object Detection with OpenCV for ROS 2](#) (1:05) · [Line Following with OpenCV for ROS 2](#) (1:12).

Stage 2 (real hardware) — only if selected, from November 2026

KD: flight controller (the portal's diagrams use ArduPilot vocabulary, while the theme names Betaflight, so confirm later), RC transmitter driven by the laptop, overhead WhyCon camera, LiPo and propeller safety. PB: ESP32, TB6612FNG, N20 encoder motors, VL53L1X ToF, MPU6050 IMU. Your breadboard skills and hardware-security interest (radio link, MQTT broker exposure) fit here, *after* the competition work is solid.

7. Learning log — writing it down counts as explaining it

Date	Module · experiment	I predicted	What happened	Why I think it happened	Still confused about

Tuning log (M3 / M6 / M7 / P3)

Date	Axis	Kp	Ki	Kd	What I saw

Answers

M1-e — One cell is $900 / 12 = 75$ px, so interior line k ($k = 1 \dots 11$) sits at $75 \times k$ px. C is the 3rd column $\rightarrow x = 225$; row 2 $\rightarrow y = 150$. C2 = (225, 150).

M2-b — Hover needs $14.7 / 21.9 \approx 67\%$. Height is lost past about 48° , since $\cos \theta = 14.7 / 21.9$.

M3-d — With $K_i = 0.5$ it creeps toward 1 instead of growing (0.5, 0.75, 0.875...); with $K_i = 2$ it explodes (2, 6, 14...). The whole running total gets multiplied every step. The standard form is $I_{\text{term}} = I_{\text{term}} + K_i \cdot \text{error}$.

M7-a — Thrust must enter as a force rotated by the attitude. About hover (thrust $\approx mg$, small angles): $x'' \approx g \cdot \theta$, $y'' \approx -g \cdot \phi$, $z'' \approx \Delta T / m$ (signs depend on convention). A 5° tilt (0.087 rad) gives about 0.86 m/s^2 sideways.